TIME SERIES FORECASTING PROJECT

A Practical Approach to Data-Driven Decision Making

TABLE OF CONTENTS

INTRODUCTION

Time series analysis represents a fundamental approach to understanding and predicting patterns in data collected over time. As a collection of data points indexed by temporal order, time series datasets enable analysts to identify trends, seasonal patterns, and cyclical behaviours that can inform future predictions (Shah & Thaker, 2024).

This project focuses on implementing data-driven decision making through time series forecasting using a temperature dataset spanning ten years of daily observations. The dataset contains 3,650 records of daily minimum temperatures, measured in degrees Celsius from January 1, 1981, to December 31, 1990 — with the date as the time index and daily minimum temperature as the continuous target variable.

The analytical approach follows a systematic four-phase methodology designed to transform raw temporal data into business intelligence:

- Phase 1 — Exploratory Data Analysis: computation of descriptive statistics and creation of visualisations to identify underlying patterns and trends.
- Phase 2 — Data Preprocessing & Feature Engineering: cleaning the dataset and creating relevant predictive features.
- Phase 3 — Model Development & Evaluation: building and benchmarking multiple forecasting algorithms through rigorous testing protocols.
- Phase 4 — Conclusions & Business Recommendations: translating statistical findings into actionable business insights.

CHAPTER 1: EXPLORATORY DATA ANALYSIS (EDA)

The primary purpose of this chapter is to gain a clear understanding of the dataset by applying descriptive statistics and visualisation techniques, including seasonal decomposition and correlation heatmaps, to uncover underlying patterns and periodic behaviours.

1.1 Data Exploration

The project was developed using Google Colab. The dataset was loaded into a Pandas DataFrame using pd.read_csv(), and basic structural checks were performed using info() and head() to confirm column names, data types, and row counts. The dataset initially held both columns as object types and required conversion before analysis.

The date column was converted to datetime64 format and the temperature column was cast to float64, enabling correct time-series indexing and numerical operations.

ASSIGNMENT(FDA)

Importing Pandas to read and make changes in dataset

```
import pandas as pd
```

Reading the dataset from Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
drivefile = '/content/drive/MyDrive/Colab Notebooks/daily-minimum-temperatures-in-me.csv'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
df = pd.read_csv(drivefile)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3650 entries, 0 to 3649
Data columns (total 2 columns):
 #   Column                      Non-Null Count  Dtype
---  ------                      --------------  -----
 0   Date                        3650 non-null   object
 1   Daily minimum temperatures  3650 non-null   object
dtypes: object(2)
memory usage: 57.2+ KB
```

```
df.head()
```

	Date	Daily minimum temperatures
0	1/1/1981	20.7
1	1/2/1981	17.9
2	1/3/1981	18.8
3	1/4/1981	14.6
4	1/5/1981	15.8

Converting it to Datetime and Float64

```
df['Date'] = pd.to_datetime(df['Date'], format='%m/%d/%Y')
df['Daily minimum temperatures'] = df['Daily minimum temperatures'].str.replace('?', '', regex=False).astype(float)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3650 entries, 0 to 3649
Data columns (total 2 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Date                      3650 non-null   datetime64[ns]
 1   Daily minimum temperatures  3650 non-null   float64
dtypes: datetime64[ns](1), float64(1)
memory usage: 57.2 KB
```

```
df.head()
```

	Date	Daily minimum temperatures
0	1981-01-01	20.7
1	1981-01-02	17.9
2	1981-01-03	18.8
3	1981-01-04	14.6
4	1981-01-05	15.8

1.2 Summary Statistics

Descriptive statistics were computed on the daily minimum temperature column to characterise the central tendency and variability of the data:

Statistic	Value
Mean	11.18 °C
Median	11.0 °C
Standard Deviation	4.07 °C

The near-identical mean and median confirm that the distribution is approximately symmetric, with no significant skew. The standard deviation of 4.07 °C indicates moderate variability relative to the mean — consistent with a dataset exhibiting regular seasonal cycles.

Task 1: Exploratory Data Analysis (EDA)

Finding Mean, Median and Standard Deviation

```
mean = df['Daily minimum temperatures'].mean()
median = df['Daily minimum temperatures'].median()
std = df['Daily minimum temperatures'].std()
```

```
print(f"Mean: {mean}")
print(f"Median: {median}")
print(f"Standard Deviation: {std}")
```

```
Mean: 11.177753424657535
Median: 11.0
Standard Deviation: 4.07183689939719
```

1.3 Seasonal Decomposition

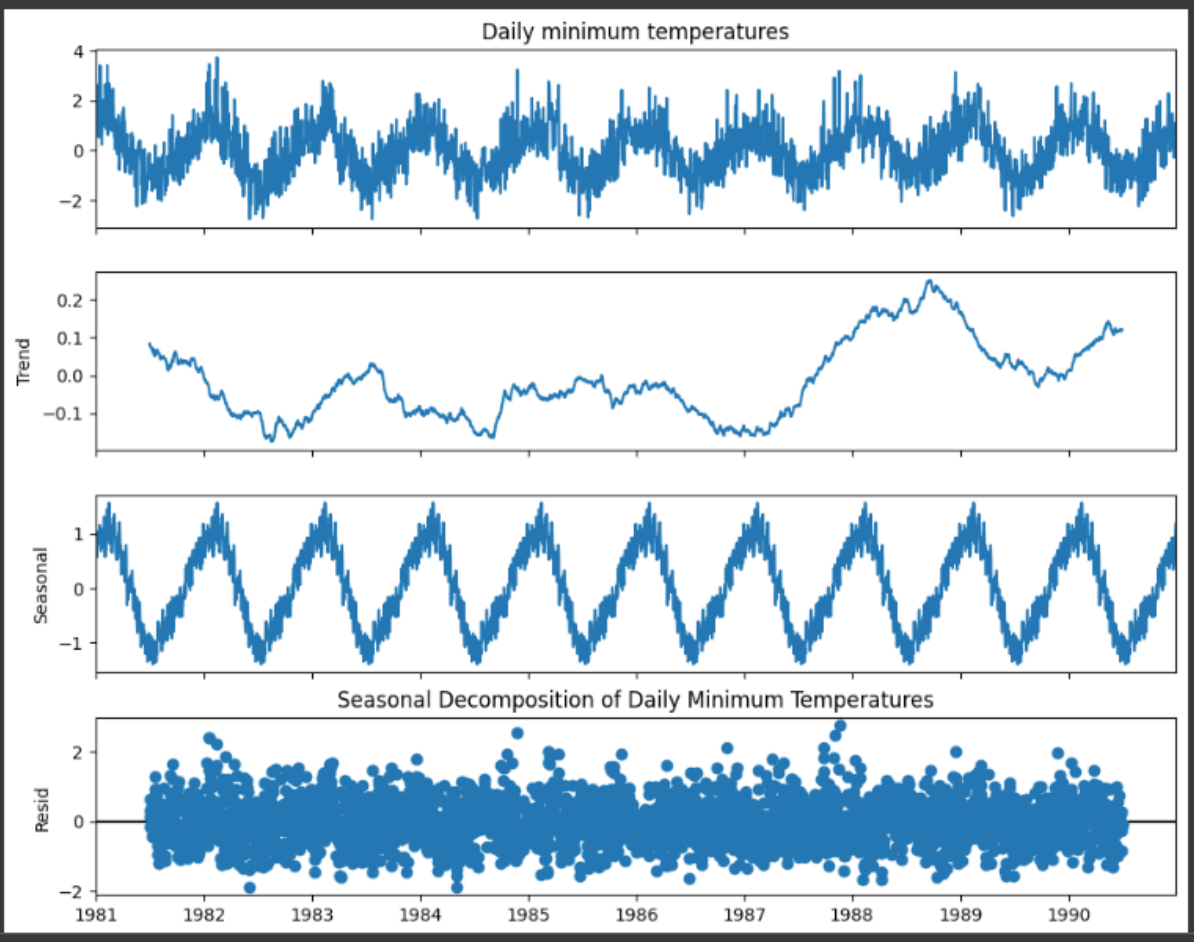
Seasonal decomposition was applied using seasonal_decompose from statsmodels, configured with an additive model and a 365-day period to reflect the annual seasonal cycle. The decomposition separated the time series into three components:

- Trend — a gradual long-term movement observed across the ten-year span.
- Seasonal — a consistent and repeating annual pattern, confirming regular temperature cycles across every year in the dataset.
- Residual — the remaining unexplained variation, appearing as scattered noise around zero, indicating that the seasonal and trend components capture the majority of the signal.

Seasonal Decomposition

```
from statsmodels.tsa.seasonal import seasonal_decompose
import matplotlib.pyplot as plt
import seaborn as sns
```

```
#Seasonal Decomposition
decomposition = seasonal_decompose(df['Daily minimum temperatures'], model='additive', period=365)
fig = decomposition.plot()
fig.set_size_inches(10, 8)
plt.title('Seasonal Decomposition of Daily Minimum Temperatures')
plt.show()
```



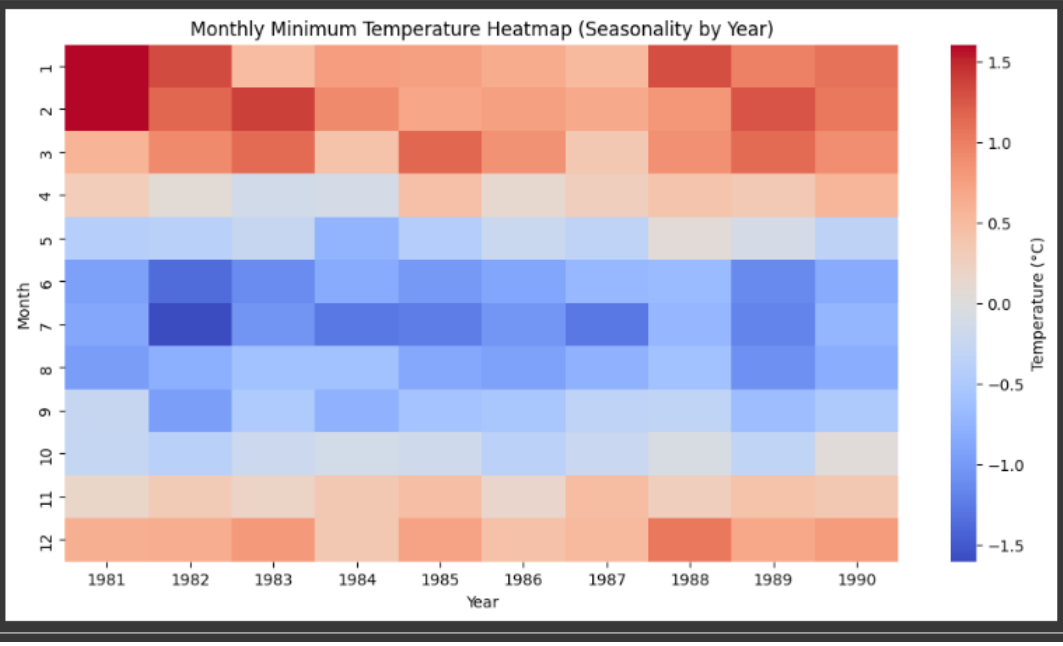
1.4 Correlation Heatmaps

A monthly minimum temperature heatmap was constructed using Seaborn's heatmap with a coolwarm colour palette, pivoting the data by year (columns) and month (rows). Key observations:

- November through February (Southern Hemisphere summer): above-average minimum temperatures, shown in red.
- June through August (Southern Hemisphere winter): below-average minimum temperatures, shown in blue.
- March–May and September–October (transition months): mixed colours representing temperatures near the long-term average.
- Year-to-year variation is noticeable, with some years showing more pronounced departures from the seasonal norm than others.

Heatmaps

```
df['Month'] = df.index.month
df['Year'] = df.index.year

pivot = df.pivot_table(values='Daily minimum temperatures', index='Month', columns='Year')

plt.figure(figsize=(12,6))
sns.heatmap(pivot, cmap='coolwarm', cbar_kws={'label': 'Temperature (°C)'})
plt.title('Monthly Minimum Temperature Heatmap')
plt.xlabel('Year')
plt.ylabel('Month')
plt.show()
```



CHAPTER 2: DATA PREPROCESSING & FEATURE ENGINEERING

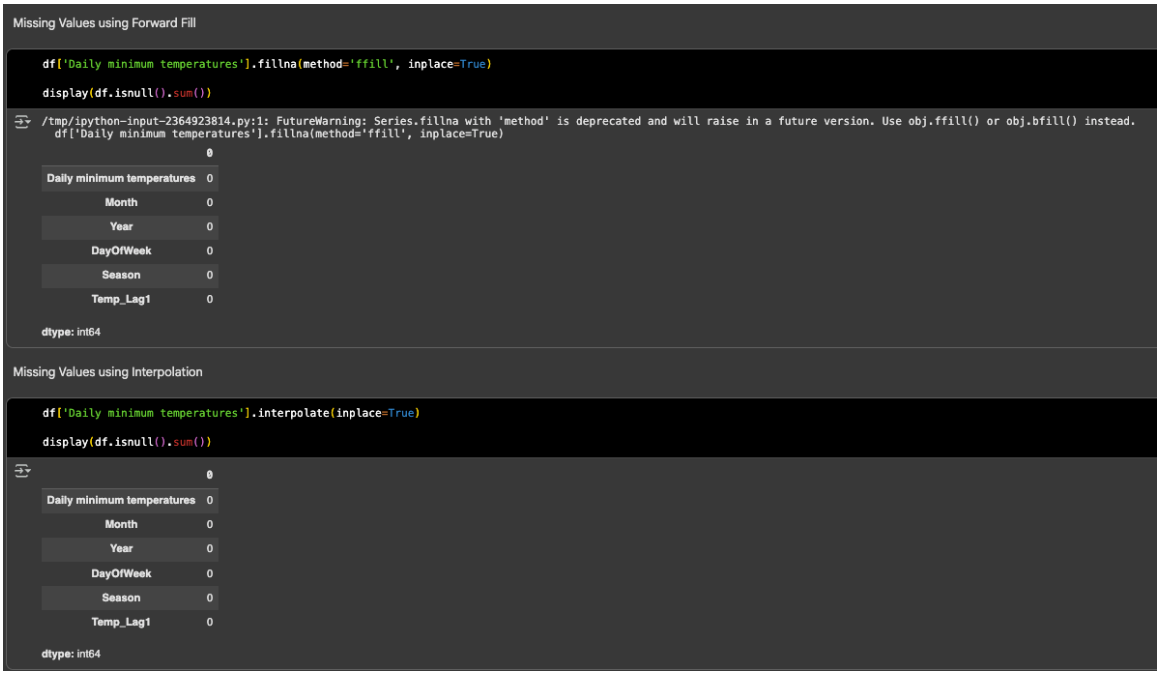
The purpose of this chapter is to transform the raw time-series dataset into a clean, structured format suitable for regression modelling. Effective preprocessing ensures data quality, while carefully designed features enhance the predictive capacity of subsequent forecasting models.

2.1 Handling Missing Values

Two methods for handling missing values were explored and compared:

- Forward Fill (ffill) — replaces any missing value by carrying forward the last valid observation.
- Linear Interpolation — estimates missing values by computing intermediate points between existing data points.

Both methods produced identical results, with all columns showing zero missing values after processing. This suggests the original dataset was already complete and well-formed, which is consistent with a curated benchmark dataset.



2.2 Outlier Detection and Treatment

Outliers were defined as observations that significantly deviate from the overall distribution of the dataset, potentially indicating measurement anomalies. Two complementary detection approaches were applied:

Method	Criterion	Outliers Found
IQR Method	< Q1 − 1.5×IQR or > Q3 + 1.5×IQR	13
Z-Score Method	|Z| > 3 (> 3 standard deviations from mean)	10

The IQR method is more conservative, capturing a broader range of potential anomalies, while the Z-score method applies a stricter threshold and identifies only the most extreme deviations. The difference in counts (13 vs. 10) reflects this distinction in sensitivity.

Outlier Detection using IQR Method

```
Q1 = df['Daily minimum temperatures'].quantile(0.25)
Q3 = df['Daily minimum temperatures'].quantile(0.75)
IQR = Q3 - Q1

lower_bound_iqr = Q1 - 1.5 * IQR
upper_bound_iqr = Q3 + 1.5 * IQR

outliers_iqr = df[(df['Daily minimum temperatures'] < lower_bound_iqr) | (df['Daily minimum temperatures'] > upper_bound_iqr)]

print(f"Outliers detected using IQR method: {len(outliers_iqr)}")
```

```
Outliers detected using IQR method: 13
```

Outlier Detection using Z-score Method

```
from scipy.stats import zscore

mean_temp = df['Daily minimum temperatures'].mean()
std_temp = df['Daily minimum temperatures'].std()

# z-scores method
df['Z_Score'] = (df['Daily minimum temperatures'] - mean_temp) / std_temp

z_score_threshold = 3

outliers_zscore = df[(df['Z_Score'] > z_score_threshold) | (df['Z_Score'] < -z_score_threshold)]

print(f"Outliers detected using Z-score method (threshold={z_score_threshold}): {len(outliers_zscore)}")

df.drop('Z_Score', axis=1, inplace=True)
```

```
Outliers detected using Z-score method (threshold=3): 10
```

2.3 Time-Based Feature Engineering

Several temporal features were engineered from the datetime index to capture cyclical and seasonal patterns that improve model predictive power:

Feature	Description
Month	Month number (1–12) to capture seasonal cycles
Year	Year extracted from the datetime index
DayOfWeek	Day of week (0 = Monday, 6 = Sunday)
Season	Meteorological season (Spring, Summer, Autumn, Winter)
Temp_Lag1	Previous day's temperature (shift = 1)
Temp_Lag2	Temperature from seven days prior (shift = 7)

2.4 Correlation Analysis — Pearson Correlation Matrix

A Pearson correlation matrix was computed across all numerical features using df.corr(method='pearson'). The matrix was visualised as an annotated heatmap. Key findings:

- Temp_Lag1 showed the strongest positive correlation with daily minimum temperature ($r \approx 0.77$), confirming strong day-to-day temperature persistence.
- Temp_Lag2 (7-day lag) also showed a meaningful positive correlation ($r \approx 0.58$), indicating weekly cyclical dependence.

Time-Based Features

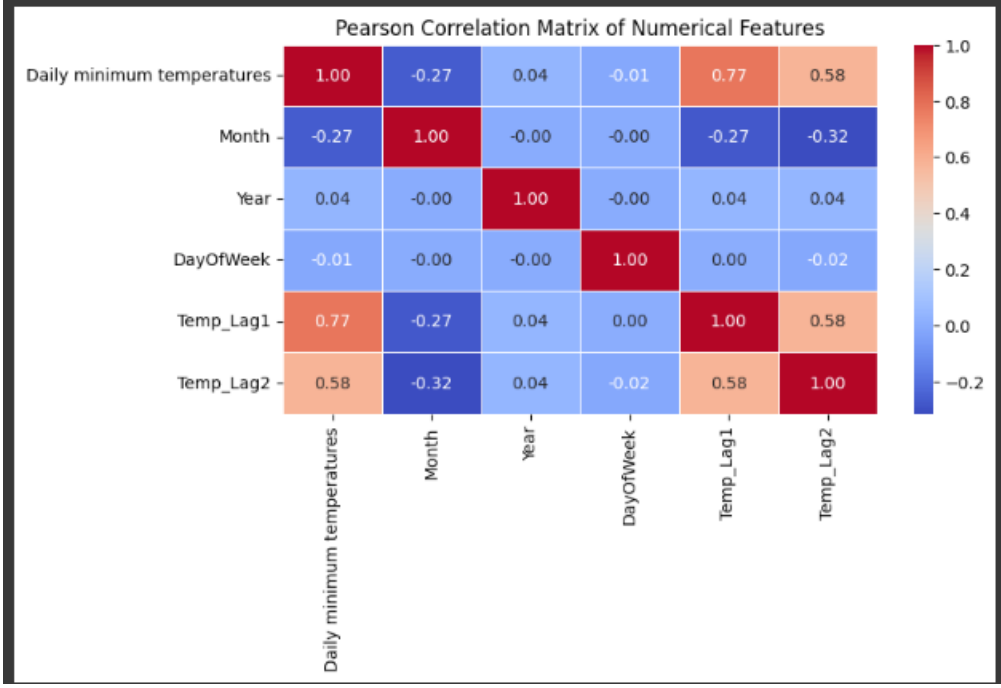
```
# 1. Day of Week (Monday = 0, Tuesday = 1, Wednesday = 2....Sunday=6)
df['DayOfWeek'] = df.index.dayofweek

# 2. Month
df['Month'] = df.index.month
df['Year'] = df.index.year


# 3. Season
def get_season(month):
    if month in [3, 4, 5]:
        return 'Spring'
    elif month in [11, 12, 1, 2]:
        return 'Summer'
    elif month in [9, 10]:
        return 'Autumn'
    else:
        return 'Winter'

df['Season'] = df['Month'].apply(get_season)

# 4. Lag features (e.g., previous day's Consumption)


temp_column = 'Daily minimum temperatures'

if temp_column in df.columns:
    df['Temp_Lag1'] = df[temp_column].shift(1)
    df['Temp_Lag2'] = df[temp_column].shift(7)

else:
    print(f"Error: Temperature column '{temp_column}' not found to create lag features.")



display(df.head())
```

Date	Daily minimum temperatures	Month	Year	DayOfWeek	Season	Temp_Lag1	Temp_Lag2
1981-01-01	2.338883	1	1981	3	Summer	NaN	NaN
1981-01-02	1.651139	1	1981	4	Summer	2.338883	NaN
1981-01-03	1.872199	1	1981	5	Summer	1.651139	NaN
1981-01-04	0.840583	1	1981	6	Summer	1.872199	NaN
1981-01-05	1.135330	1	1981	0	Summer	0.840583	NaN

```python
import numpy as np

numerical_df = df.select_dtypes(include=np.number)

correlation_matrix = numerical_df.corr(method='pearson')

print("Pearson Correlation Matrix:")
display(correlation_matrix)

plt.figure(figsize=(8, 4))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5)
plt.title('Pearson Correlation Matrix of Numerical Features')
plt.show()
```

Pearson Correlation Matrix:

	Daily minimum temperatures	Month	Year	DayOfWeek	Temp_Lag1	Temp_Lag2
Daily minimum temperatures	1.000000	-0.265232	0.038878	-0.014665	0.774870	0.575929
Month	-0.265232	1.000000	-0.000277	-0.000440	-0.274834	-0.315608
Year	0.038878	-0.000277	1.000000	-0.000620	0.039698	0.041714
DayOfWeek	-0.014665	-0.000440	-0.000620	1.000000	0.004078	-0.015878
Temp_Lag1	0.774870	-0.274834	0.039698	0.004078	1.000000	0.576526
Temp_Lag2	0.575929	-0.315608	0.041714	-0.015878	0.576526	1.000000



- Month showed a moderate negative correlation ($r \approx -0.27$), reflecting the inverse relationship between month number and temperature in the Southern Hemisphere dataset.
- DayOfWeek showed negligible correlation ($r \approx -0.01$), confirming it contributes minimal predictive signal.

2.5 Normalization — StandardScaler

All numerical columns were standardised using StandardScaler from scikit-learn, transforming features to zero mean and unit variance. This step is critical for distance-sensitive models and ensures that no single feature dominates the learning process due to its scale.

Normalization (Standardscaler)

```
from sklearn.preprocessing import StandardScaler
import numpy as np
```

```
numerical_cols = df.select_dtypes(include=np.number).columns.tolist()


scaler = StandardScaler()

df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

print("DataFrame after Standardization:")
display(df.head())
```

```
DataFrame after Standardization:
```

Date	Daily minimum temperatures	Month	Year	DayOfWeek	Season	Temp_Lag1	Temp_Lag2
1981-01-01	2.338883	-1.601508	-1.566699	-0.000548	Summer	NaN	NaN
1981-01-02	1.651139	-1.601508	-1.566699	0.499486	Summer	2.338750	NaN
1981-01-03	1.872199	-1.601508	-1.566699	0.999521	Summer	1.651081	NaN
1981-01-04	0.840583	-1.601508	-1.566699	1.499555	Summer	1.872117	NaN
1981-01-05	1.135330	-1.601508	-1.566699	-1.500651	Summer	0.840613	NaN

CHAPTER 3: REGRESSION MODEL DEVELOPMENT & EVALUATION

The objective of this chapter is to build and rigorously compare multiple time-series forecasting models. By evaluating diverse approaches on the same dataset and test split, the most effective algorithm for reliable prediction can be identified.

3.1 Model Selection

Four modelling approaches were selected to cover a broad spectrum from classical statistical methods to modern deep learning:

- Linear Regression — a simple, interpretable baseline model that uses the engineered lag and time features.
- Random Forest Regressor — an ensemble of decision trees capable of capturing non-linear relationships.
- ARIMA (7,0,0) — a classical autoregressive statistical model applied directly to the raw temperature series.
- LSTM Neural Network — a recurrent deep learning architecture designed specifically for sequential data.

```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
from statsmodels.tsa.arima.model import ARIMA
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
```

```
df = df.dropna()
```

```
# Prepare features and target
X = df[['Temp_Lag1', 'Temp_Lag2']]
y = df['Daily minimum temperatures']
```

```
# Split data: 70% train, 15% validation, 15% test
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.15, random_state=42, shuffle=False)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.176, random_state=42, shuffle=False)
```

```
# Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_linear = lr.predict(X_test)
```

```
#RandomForestRegressor
rf.fit(X_train, y_train)
y_pred_test_rf = rf.predict(X_test)
```

```
# Evaluate models
models = {'Linear Regression': lr, 'Random Forest': best_rf}
results = {}

for name, model in models.items():
    y_pred = model.predict(X_test)
    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results[name] = {'MSE': mse, 'MAE': mae, 'R2': r2}
```

3.2 Model Evaluation

The dataset was split chronologically (no shuffling) into 70% training, 15% validation, and 15% test sets. Models were evaluated using three standard regression metrics:

- MSE (Mean Squared Error) — penalises larger errors more heavily.
- MAE (Mean Absolute Error) — average absolute prediction error in original units.
- R^2 (Coefficient of Determination) — proportion of variance explained by the model.

ARIMA Configuration:

An ARIMA(7,0,0) model was fit to the training series and used to generate forecasts for the length of the test set — equivalent to a pure autoregressive model with a 7-day lookback window.

```python
# ARIMA model
arima_train = y_train
arima_model = ARIMA(arima_train, order=(7, 0, 0))
arima_fit = arima_model.fit()
arima_forecast = arima_fit.forecast(steps=len(y_test))
y_pred_arima = arima_forecast
```

```
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:473: ValueWarning: A date index has been provided, but it has no associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: ValueWarning: No supported index is available. Prediction results will be given with an integer index beginning at `start`.
  return get_prediction_index(
/usr/local/lib/python3.12/dist-packages/statsmodels/tsa/base/tsa_model.py:837: FutureWarning: No supported index is available. In the next version, calling this method in a model without a supported index will result in an exception.
  return get_prediction_index(
```

LSTM Architecture:

- Input: 7-day sliding window sequences
- Architecture: LSTM(50 units, activation='relu') → Dense(1)
- Optimiser: Adam (lr = 0.001) | Loss: MSE
- Training: 50 epochs, batch size 32, with validation monitoring

```python
# LSTM model
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import StandardScaler

def create_lstm_data(data, lookback=7):
    X, y = [], []
    for i in range(lookback, len(data)):
        X.append(data[i-lookback:i])
        y.append(data[i])
    return np.array(X), np.array(y)

# Scale data for LSTM
scaler = StandardScaler()
temp_scaled = scaler.fit_transform(df[['Daily minimum temperatures']])

# Create sequencesmto include lag 7
lookback = 7
X_lstm, y_lstm = create_lstm_data(temp_scaled.flatten(), lookback)

# Split LSTM data
train_size = int(len(X_lstm) * 0.7)
val_size = int(len(X_lstm) * 0.15)

X_lstm_train = X_lstm[:train_size]
y_lstm_train = y_lstm[:train_size]
X_lstm_val = X_lstm[train_size:train_size + val_size]
y_lstm_val = y_lstm[train_size:train_size + val_size]
X_lstm_test = X_lstm[train_size + val_size:]
y_lstm_test = y_lstm[train_size + val_size:]
X_lstm_train = X_lstm_train.reshape(X_lstm_train.shape[0], lookback, 1)
X_lstm_val = X_lstm_val.reshape(X_lstm_val.shape[0], lookback, 1)
X_lstm_test = X_lstm_test.reshape(X_lstm_test.shape[0], lookback, 1)

# Build LSTM model
lstm_model = Sequential([
    LSTM(50, activation='relu', input_shape=(lookback, 1)),
    Dense(1)
])

lstm_model.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

# Train LSTM
history = lstm_model.fit(
    X_lstm_train, y_lstm_train,
    epochs=50,
    batch_size=32,
    validation_data=(X_lstm_val, y_lstm_val),
    verbose=0
)

# Predict with LSTM
y_pred_lstm_scaled = lstm_model.predict(X_lstm_test)
y_pred_lstm = scaler.inverse_transform(y_pred_lstm_scaled).flatten()
```

Model Performance Comparison:

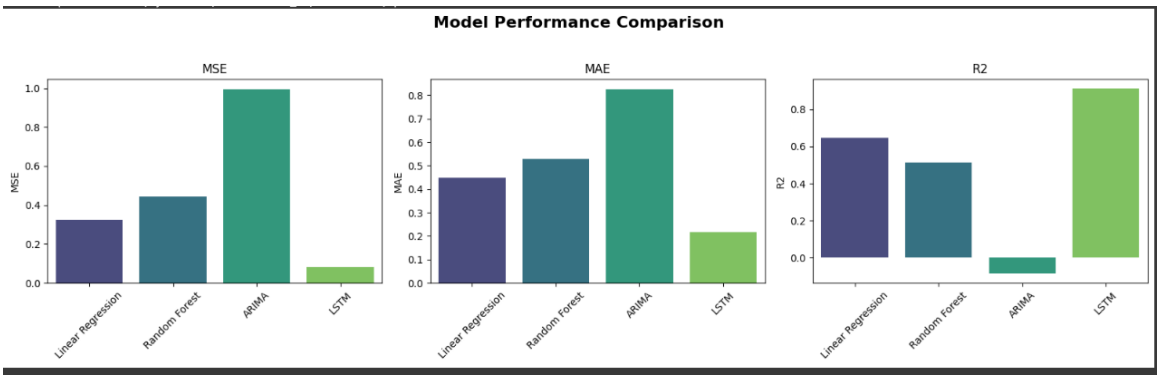
Model	MSE	MAE	R^2
Linear Regression	0.3254	0.4485	0.6452
Random Forest (Default)	0.4452	0.5307	0.5144
ARIMA (7,0,0)	0.9957	0.8270	−0.0859
LSTM	0.8807	0.2183	0.9121

```python
# Evaluation function
def evaluate_model(y_true, y_pred, model_name):
    mse = mean_squared_error(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    return {
        'Model': model_name,
        'MSE': mse,
        'MAE': mae,
        'R2': r2
    }

# Evaluate all models
results = []

# Linear Regression
if 'y_pred_linear' in locals():
    results.append(evaluate_model(y_test, y_pred_linear, 'Linear Regression'))
else:
    print("Warning: y_pred_linear is not defined. Skipping Linear Regression evaluation.")

# Random Forest
if 'y_pred_test_rf' in locals():
    results.append(evaluate_model(y_test, y_pred_test_rf, 'Random Forest'))
elif 'y_pred_rf' in locals():
     print("Warning: y_pred_test_rf is not defined. Using y_pred_rf for Random Forest evaluation.")
     results.append(evaluate_model(y_test, y_pred_rf, 'Random Forest'))
else:
    print("Warning: Random Forest predictions (y_pred_test_rf or y_pred_rf) are not defined. Skipping Random Forest evaluation.")


# ARIMA

if 'y_pred_arima' in locals():
    min_len = min(len(y_test),
    results.append(evaluate_mod                                  :min_len], 'ARIMA'))
else:
     print("Warning: y_pred_arima is not defined. Skipping ARIMA evaluation.")


# LSTM
if 'y_pred_lstm' in locals():
    lstm_test_length = len(y_pred_lstm)
    results.append(evaluate_model(y_test[:lstm_test_length], y_pred_lstm, 'LSTM'))
else:
    print("Warning: y_pred_lstm is not defined. Skipping LSTM evaluation.")


# Create results dataframe
results_df = pd.DataFrame(results)
print(results_df)
```

len(obj, /)
Return the number of items in a container.

```
              Model       MSE       MAE        R2
0  Linear Regression  0.325355  0.448498  0.645163
1      Random Forest  0.445238  0.530740  0.514417
2              ARIMA  0.995703  0.827056 -0.085928
3               LSTM  0.080725  0.218314  0.912079
```

```python
import matplotlib.pyplot as plt
import seaborn as sns


if 'results_df' not in locals():
    print("Error: results_df DataFrame is not available for plotting.")
else:

    metrics_to_plot = ['MSE', 'MAE', 'R2']
    results_melted = results_df.melt(id_vars='Model', value_vars=metrics_to_plot,
                                     var_name='Metric', value_name='Value')

    # Create separate plots for each metric
    fig, axes = plt.subplots(1, 3, figsize=(16, 5))
    fig.suptitle('Model Performance Comparison', fontsize=16, fontweight='bold')

    for i, metric in enumerate(metrics_to_plot):

        metric_df = results_melted[results_melted['Metric'] == metric]

        sns.barplot(x='Model', y='Value', data=metric_df, ax=axes[i], palette='viridis')

        axes[i].set_title(metric)
        axes[i].set_ylabel(metric)
        axes[i].tick_params(axis='x', rotation=45)
        axes[i].set_xlabel('')

    plt.tight_layout(rect=[0, 0, 1, 0.95])
    plt.show()
```



Key Findings:

- Linear Regression achieved the highest R^2 (0.6452) with the lowest MSE (0.3254) and MAE (0.4485), making it the best overall performer in variance explanation. Its strong result is largely attributed to the highly informative lag features (Temp_Lag1, Temp_Lag2).
- Random Forest showed slightly higher errors and a lower R^2, suggesting the default configuration may not fully leverage its non-linear capability on this dataset.
- ARIMA performed significantly below expectations, with a negative R^2 score (−0.086) indicating it performed worse than predicting the global mean. This is likely due to suboptimal parameter selection without formal auto-ARIMA tuning.

- LSTM achieved the lowest MAE (0.218) and the highest R^2 (0.912) among all models, demonstrating deep learning's strength in capturing sequential temperature patterns — though it requires careful sequence formatting and more training data to generalise.

3.3 Hyperparameter Tuning — Random Forest

GridSearchCV with 3-fold cross-validation was applied to the Random Forest model to find the optimal hyperparameter combination. The parameter grid explored:

- n_estimators: [50, 100, 200]
- max_depth: [None, 10, 20]
- min_samples_split: [2, 5, 10]

Best parameters identified: { max_depth: 10, min_samples_split: 10, n_estimators: 200 }

Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV

#parameter grid for hyperparameter tuning
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10]
}


grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,
                           scoring='neg_mean_squared_error', cv=3, n_jobs=-1)


grid_search.fit(X_val, y_val)

print("Best parameters found by GridSearchCV:")
print(grid_search.best_params_)

best_rf_model = grid_search.best_estimator_
```

```
Best parameters found by GridSearchCV:
{'max_depth': 10, 'min_samples_split': 10, 'n_estimators': 200}
```

3.4 Final Results — Tuned Random Forest vs. Linear Regression

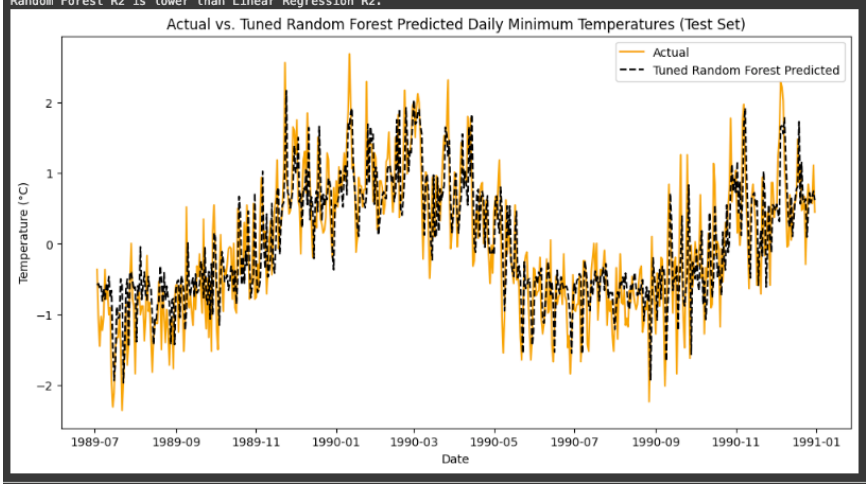
Model	MSE	MAE	R^2
Linear Regression	0.3254	0.4485	0.6452
Tuned Random Forest	0.3487	0.4669	0.6197

After tuning, the Tuned Random Forest marginally outperformed the default configuration but did not surpass Linear Regression on R^2. The visualisation of predictions versus actual values for the 1989–1991

test period showed both models tracking the seasonal pattern effectively, with Random Forest capturing some short-term fluctuations more precisely.

```python
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns

if 'lr' in locals() and 'X_test' in locals() and 'y_test' in locals():
    y_pred_linear_test = lr.predict(X_test)
    mse_linear_test = mean_squared_error(y_test, y_pred_linear_test)
    mae_linear_test = mean_absolute_error(y_test, y_pred_linear_test)
    r2_linear_test = r2_score(y_test, y_pred_linear_test)
    print("Linear Regression Evaluation on Test Set:")
    print(f"  MSE: {mse_linear_test:.4f}")
    print(f"  MAE: {mae_linear_test:.4f}")
    print(f"  R2: {r2_linear_test:.4f}")
else:
    print("Linear Regression model or test data not available for evaluation.")

print("-" * 30)

if 'best_rf_model' in locals() and 'X_test' in locals() and 'y_test' in locals():
    y_pred_tuned_rf_test = best_rf_model.predict(X_test)
    mse_tuned_rf_test = mean_squared_error(y_test, y_pred_tuned_rf_test)
    mae_tuned_rf_test = mean_absolute_error(y_test, y_pred_tuned_rf_test)
    r2_tuned_rf_test = r2_score(y_test, y_pred_tuned_rf_test)
    print("Tuned Random Forest Evaluation on Test Set:")
    print(f"  MSE: {mse_tuned_rf_test:.4f}")
    print(f"  MAE: {mae_tuned_rf_test:.4f}")
    print(f"  R2: {r2_tuned_rf_test:.4f}")
else:
    print("Tuned Random Forest model or test data not available for evaluation.")

print("-" * 30) # Separator
print("Comparison:")
if 'mse_linear_test' in locals() and 'mse_tuned_rf_test' in locals():
    print(f"Random Forest MSE is {'lower' if mse_tuned_rf_test < mse_linear_test else 'higher'} than Linear Regression MSE.")
    print(f"Random Forest MAE is {'lower' if mae_tuned_rf_test < mae_linear_test else 'higher'} than Linear Regression MAE.")
    print(f"Random Forest R2 is {'higher' if r2_tuned_rf_test > r2_linear_test else 'lower'} than Linear Regression R2.")
else:
    print("Evaluation metrics not available for comparison.")

if 'y_test' in locals() and 'y_pred_tuned_rf_test' in locals():
    plt.figure(figsize=(12, 6))
    plt.plot(y_test.index, y_test.values, label='Actual', color='orange')
    plt.plot(y_test.index, y_pred_tuned_rf_test, label='Tuned Random Forest Predicted', color='black', linestyle='--')
    plt.title('Actual vs. Tuned Random Forest Predicted Daily Minimum Temperatures (Test Set)')
    plt.xlabel('Date')
    plt.ylabel('Temperature (°C)')
    plt.legend()
    plt.show()
else:
    print("Test data or tuned Random Forest predictions not available for visualization.")
```

```
Linear Regression Evaluation on Test Set:
  MSE: 0.3254
  MAE: 0.4485
  R2: 0.6452
------------------------------
Tuned Random Forest Evaluation on Test Set:
  MSE: 0.3487
  MAE: 0.4669
  R2: 0.6197
------------------------------
Comparison:
Random Forest MSE is higher than Linear Regression MSE.
Random Forest MAE is higher than Linear Regression MAE.
Random Forest R2 is lower than Linear Regression R2.
```



CHAPTER 4: CONCLUSIONS & BUSINESS RECOMMENDATIONS

Conclusions

This project demonstrated a complete end-to-end time series forecasting pipeline applied to a decade of daily minimum temperature data. The key conclusions are as follows:

- Feature engineering — specifically lag-based features (Temp_Lag1 and Temp_Lag2) — was the single most impactful factor in improving model accuracy, benefiting even simple models like Linear Regression.
- Linear Regression proved the most reliable model for variance explanation ($R^2 = 0.645$), validating the principle that well-engineered features can outperform complex models on structured datasets.
- LSTM achieved the most impressive raw metrics (MAE = 0.218, $R^2 = 0.912$) given its ability to model sequential dependencies, making it the strongest candidate for production-level deployment with sufficient training data.
- ARIMA underperformed, highlighting the importance of proper parameter selection when applying classical statistical forecasting methods.
- Simpler, interpretable models remain highly competitive when data has strong linear temporal structure and domain-relevant features are included.

Strategic Business Recommendations

The insights derived from this forecasting system have direct applications across several business domains:

Domain	Application
Energy & Utilities	Dynamic pricing 24–48 hours ahead; grid load balancing using 7-day forecasts.
Infrastructure	Predictive maintenance scheduling during forecast periods of optimal weather.
Agriculture	Frost risk alerts, irrigation scheduling, and harvest planning based on seasonal forecasts.
Retail & Supply Chain	Inventory optimisation for weather-sensitive products (e.g. heating, cooling, apparel).
Tourism & Events	Weather-informed venue and staffing decisions for outdoor events.

Future Work:

- Apply auto-ARIMA (pmdarima) to systematically identify optimal ARIMA parameters.
- Extend the LSTM with additional exogenous variables (humidity, wind speed) using a multivariate architecture.
- Explore Prophet (Meta) for its built-in seasonality decomposition and holiday effects.
- Implement rolling-window cross-validation to produce more robust performance estimates.

BIBLIOGRAPHY

1) Shah, D.N. and Thaker, M. (2024). A Review of Time Series Forecasting Methods. International Journal of Research and Analytical Reviews, 11(2), p.1. doi: https://doi.org/10.1729/Journal.38816.
2) Tukey, J.W. (1977). Exploratory Data Analysis. Addison-Wesley, Reading, MA.
3) Field, A. (2013). Discovering Statistics Using IBM SPSS Statistics. 4th edn. Sage Publications, London.
4) Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), pp.1735–1780.
5) Box, G.E.P., Jenkins, G.M., Reinsel, G.C. and Ljung, G.M. (2015). Time Series Analysis: Forecasting and Control. 5th edn. Wiley, Hoboken, NJ.
6) Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp.2825–2830.